

RootNode

-

Exploitable vulnerabilities and answers

31 OCTOBER

V1.0

Munk IT

Author: Luke de Munk

Munk IT

Contents

CONTENTS **2**

1. INTRODUCTION **3**

2. GOALS **4**

3. VULNERABILITIES **5**

3.1 WEAK, GUESSABLE OR HARDCODED PASSWORDS 5

3.1.1 Brute force passwords - Medium *5*

3.1.2 Serial CLI user leak - Easy *5*

3.1.3 Serial web user leak - Medium *6*

3.2 INSECURE NETWORK SERVICES 7

3.2.1 Bypass role based hidden UI elements - Easy *7*

3.2.2 Man-in-the-middle - Medium *7*

3.2.3 Cross-site request forgery - Medium *8*

3.2.4 Botnet scripting - Medium *8*

3.2.5 DoS - Medium *8*

3.3 INSECURE ECOSYSTEM SERVICES 10

3.3.1 Encryption leak - Medium *10*

3.3.2 Buffer overflow - Hard *10*

3.4 INSECURE UPDATE MECHANISM 12

3.4.1 Exploitable OTA update *12*

3.4.2 Non-volatile data reading (by altered firmware) - Hard *12*

3.5 INSUFFICIENT PRIVACY PROTECTION 13

3.6 INSECURE DATA TRANSFER AND STORAGE 14

3.6.1 Exploitable file upload & download - Medium *14*

3.6.2 Path traversal *14*

3.7 LACK OF DEVICE MANAGEMENT 15

3.7.1 X *15*

3.8 INSECURE DEFAULT SETTINGS 16

3.9 LACK OF PHYSICAL HARDENING 16

3.9.1 Firmware dump - Hard *16*

1. Introduction

This teacher manual supports instructors in explaining and demonstrating the security vulnerabilities included in the RootNode project. The exercises are based primarily on the OWASP Internet of Things Top 10 (2018), supplemented by several additional vulnerabilities commonly found in embedded systems, web applications, network services, and device firmware.

For each exercise, this manual describes the learning objectives, the relevant vulnerability, and the steps required to exploit it and complete the assignment. It is intended to help instructors prepare lessons, demonstrate attacks in a controlled environment, and assist trainees when they encounter difficulties.

Technical instructions for installing, configuring, and operating the hardware and firmware are available in the **README.md**. A separate **Goals.pdf** contains the learning objectives.

The device is intentionally vulnerable and should only be used in an isolated, authorized training environment. The techniques described must not be applied to systems without explicit permission.

If you discover additional vulnerabilities that are not documented in this manual, please report them to Luke de Munk so they can be reviewed and, where appropriate, incorporated into the project documentation and training material.

2. Goals

A list of objectives provides a convenient way to track progress while exploring and exploiting the vulnerabilities in this project. Some objectives can be achieved in multiple ways, encouraging to investigate different attack paths. This chapter presents only the objectives. The exploitable vulnerabilities and the methods used to exploit them are described in the following chapter.

1. **Gain access to the power and brightness controls**
2. **Gain access to the serial CLI**
3. **Log in to the web interface as a regular user**
4. **Gain administrator access to the web interface**
5. **Download files containing sensitive information**
6. **Gain root privileges in the serial CLI**
7. **Obtain the credentials of the configured web users**
8. **Modify the web-user configuration by uploading a manipulated file**
9. **Automate the control of multiple devices on the same network**
10. **Install or execute unauthorized firmware on the device**
11. **Buffer overflow**

3. Vulnerabilities

This chapter explains how the vulnerabilities and security weaknesses of the RootNode can be exploited. Perform all exercises only on the supplied RootNode and an isolated training network. Do not run the network attacks against systems or networks for which permission has not been given.

3.1 Weak, Guessable or Hardcoded Passwords

The RootNode uses several credentials for the web interface, serial-console access and the simulated root account. Weak passwords, missing rate limiting and credentials embedded in the firmware make these accounts easier to compromise than accounts on a properly secured device.

3.1.1 Brute force passwords - Medium

The login endpoint does not introduce a delay, temporary account lockout or attempt limit after an incorrect password. An attacker who knows or guesses a username can therefore submit many candidate passwords and distinguish failed attempts from a successful login by the HTTP status and JSON response. Short or common passwords can consequently be recovered through a dictionary attack.

Preparation

Connect a computer to the same isolated network as the RootNode and determine the device's IP address or hostname. A web-proxy tool such as Burp Suite can be used to capture and repeat the login request. A small, controlled password list should be used to prevent the exercise from becoming an unintended denial-of-service test.

Steps to reproduce

1. Open http://smartlight/login and capture one login request with the browser developer tools or an intercepting proxy.
2. Confirm that the request is a POST request to /login containing *username* and *password*.
3. Send several candidate passwords for one known username with a custom script, Burp Intruder or another rate-controlled request tool.
4. Compare the responses. Invalid credentials return an authentication failure, while valid credentials create a session and return an authenticated role.
5. Stop the test immediately after finding the password and record that no lockout or throttling occurred.

3.1.2 Serial CLI user leak - Easy

The startup output contains hexadecimal byte data from which the serial password can be recovered. After the console has been unlocked, it exposes a Linux-like command interface. The root-only **webusers** command displays the configured normal web-user password in clear

text, although the administrator password is masked. Debug output and administrative commands should not expose credentials on a deployed device.

Preparation
Serial access to the RootNode is required. Connect the board through USB and open its COM port with the Arduino IDE serial monitor, PuTTY or another serial terminal. Configure the connection for 115200 baud, eight data bits, no parity and one stop bit.

Steps to reproduce
1. Connect the USB cable and open the correct COM port at 115200 baud. (LF line ending)
2. Press the reset button and capture the complete startup output.
3. Convert the leaked hexadecimal bytes to characters and recover the serial password.
4. Enter **debug --on** and supply the recovered serial password when prompted.
5. You now have access to the CLI.

3.1.3 Serial web user leak - Medium
The RootNode stores its web accounts in */etc/SmartLight/webusers.db*. The same file is backed by an external file that can be downloaded by an authenticated user. Its account records are encrypted, but the ciphertext is still sensitive and can be decrypted by combining this issue with the encryption-key leak in section 3.3.1. This exposes HTTP usernames, passwords and roles outside the login process.

Preparation
Open the webinterface.

Steps to reproduce
1. Sign in to the RootNode.
2. Open the file explorer and browse to */etc/SmartLight*.
3. Download *webusers.db*, or request the same virtual path through the */download* endpoint.
4. In the serial CLI, run **cat /etc/SmartLight/webusers.db** and compare the ciphertext with the downloaded file.
5. Recover the AES key as described in section 3.3.1 and decrypt each encrypted account line.
6. Verify that the plaintext contains the username, password and numerical role of each account.

3.2 Insecure Network Services

The RootNode exposes its web interface over unencrypted HTTP. Several device-control endpoints also trust the request itself instead of consistently enforcing authentication and authorization.

3.2.1 Bypass role based hidden UI elements - Easy

The user interface hides administrative controls according to the role reported by the device. Hiding an element in HTML or JavaScript is not access control. The RootNode's power and brightness handlers do not enforce the intended administrator check, so an unauthenticated client can call the underlying endpoints directly.

Preparation

Open the webinterface.

Steps to reproduce

1. Open Element Inspect (on Chrome) and find the hidden UI elements.
2. Make UI elements visible and use them.
3. Open the browser console and see the requests the actions make. (*/set_power*, */set_brightness*)
4. Request */set_power?state=1* directly from the RootNode.
5. Request */set_brightness?brightness=512* directly from the RootNode.
6. Observe that the physical LED changes even though the controls were not available to the current role.
7. This can be used to create an automated bot script, when multiple controllers are in the network. See 3.2.4.

3.2.2 Man-in-the-middle - Medium

The web interface uses HTTP rather than HTTPS. HTTP provides neither transport encryption nor server authenticity. A party able to observe the local network can therefore read requests and responses, including login data and session information, and an active intermediary could modify commands sent to the device.

Preparation

Place the RootNode, the trainee computer and a test client on an isolated Wi-Fi network that is fully controlled by the instructor. Start a packet capture on the access point or on an approved monitoring interface. Wireshark can be used to inspect the captured traffic.

Steps to reproduce

1. Start a packet capture on the controlled training network.
2. From the test client, open http://smartlight/login and perform a login.
3. Use the Wireshark display filter *http* to locate the request to */login*.
4. Inspect the form data and subsequent request headers.
5. Confirm that the credentials and session traffic are visible as ordinary HTTP data.

3.2.3 Cross-site request forgery - Medium

The LED state is changed through a predictable HTTP GET request. The endpoint does not require a CSRF token and does not enforce authentication. A malicious page can therefore include the device URL as an image source. Merely opening the page causes the browser to send the request and change the RootNode state.

Preparation

Use the supplied *software/scripts/csrf.html* file on the isolated training network. Replace its example IP address with the address of the RootNode. Open the file only from a test computer that is allowed to control the training device.

Steps to reproduce

1. Set the RootNode LED to a known state.
2. Edit the image URL in csrf.html to http://<RootNode-IP>/set_power?state=0 or state=1.
3. Open the HTML file in a browser connected to the RootNode network.
4. Observe that the image cannot be rendered but its HTTP request is still sent.
5. Verify that the LED state changes without interaction with the RootNode interface.

3.2.4 Botnet scripting - Medium

Because the device-control endpoint is predictable and unauthenticated, the same command can be sent automatically to many RootNodes. The supplied exercise script enumerates addresses in a subnet and uses parallel HTTP requests. It demonstrates how insecure IoT devices can be controlled as a group without first compromising an account on every device.

Preparation

Use only a dedicated classroom subnet containing authorized RootNode devices. Open *software/scripts/BotLED_skeleton.py* and set its hosts variable to that subnet. Python 3 is required to run the exercise.

Steps to reproduce

1. Configure hosts with the authorized lab subnet, for example a dedicated /24 network.
2. Complete the control function so it requests http://<address>/set_power?state=<state> with a short timeout.
3. Start the script and monitor which classroom devices respond.
4. Confirm that multiple RootNodes change state from the same script.

3.2.5 DoS - Medium

The ESP8266 has limited memory and handles its web service in the main firmware loop. Many concurrent or repeated HTTP requests can consume connection slots and processing time, making the interface slow or temporarily unavailable. A sufficiently heavy test may also trigger the watchdog and restart the device, maybe that happened already when trying to brute force passwords.

Preparation

Connect one RootNode to an isolated network and keep its serial output visible. Establish a baseline by recording normal page-response time and free heap. Use a request tool with a strict request limit rather than an unlimited loop.

Steps to reproduce

1. Confirm that */get_state* responds normally and note the response time.
2. Send a small, controlled batch, such as 100 requests with five concurrent connections, to **http://<RootNode-IP>/get_state**.
3. While the batch runs, try to load the web interface from a second browser.
4. Observe increased latency, failed connections or a watchdog restart in the serial output.
5. Increase the load only gradually and stop as soon as the effect is demonstrated.
6. Allow the RootNode to recover and verify that normal operation returns.

3.3 Insecure Ecosystem Services

The serial shell and web interface expose functions that are unsafe when they are combined. Data obtained through one interface can be reused to compromise another part of the RootNode.

3.3.1 Encryption leak - Medium

There is an AES key stored in the filesystem and is readable from the serial shell. The gpg command also provides a decryption operation. Storing ciphertext and its key on the same accessible device removes most of the protection offered by encryption.

Preparation

Unlock the serial CLI and obtain the encrypted contents of *webusers.db*. The cat and gpg commands are required for this exercise.

Steps to reproduce

1. Run **cat /etc/SmartLight/aes.key** and record the 16-byte key.
2. Run **cat /etc/SmartLight/webusers.db** or use the downloaded copy from section 3.1.3.
3. Ignore the format: encrypted header and copy one encrypted line.
4. Run **gpg --decrypt "<recovered-key>" "<encrypted-line>"**.
5. Repeat the command for the remaining account line.
6. Confirm that the results follow the *username:password:role* format.

3.3.2 Buffer overflow - Hard

In the past, buffer overflows were a common software vulnerability. Nowadays, many software applications are developed using programming languages and techniques that help prevent such vulnerabilities. However, buffer overflows can still occur in embedded systems and microcontrollers, such as ESP devices.

A buffer overflow, also known as a buffer overrun, occurs when the amount of data written to a memory buffer exceeds its allocated storage capacity. This can cause the program to overwrite adjacent memory locations. For example, a buffer designed to store login credentials may allocate 8 bytes for an input. If the program receives 10 bytes, the additional 2 bytes may be written beyond the buffer's boundaries, potentially corrupting nearby memory.

If the overwritten memory contains executable code or other critical program data, the application may behave unpredictably, produce incorrect results, encounter memory access errors, or crash. In more severe cases, attackers who understand the program's memory layout may deliberately provide input that exceeds the buffer's capacity to overwrite specific memory regions. By replacing existing code or manipulating control-flow data, they may be able to execute malicious code. For instance, an attacker could overwrite a function pointer or another control-flow pointer to redirect program execution to a malicious payload, potentially gaining control over the program.

testprogram.c copies an argument into a fixed-size buffer without checking its length. The simulator models a little-endian overwrite of the saved return address. By replacing that address with the address of the login function, the CLI identity can be changed to root.

Preparation

Unlock the serial CLI and navigate to */home/user*. The vulnerable program is simulated by the RootNode; the exercise does not execute native user-provided machine code on the ESP8266.

Steps to reproduce

1. Run **cd /home/user** and inspect the source with **cat testprogram.c**.
2. Run **objdump -d testprogram** and locate the address of the login function.
3. Determine the offset between the start of the input buffer and the saved return address.
4. Supply sixteen filler bytes followed by the login address in little-endian form. For the current simulation, the command is **./testprogram AAAAAAAAAAAAAAAA\x88\x04\x01\x00**.
5. Run **whoami** or id and confirm that the shell now has root identity.
6. Run exit when the root-only exercises are complete.

3.4 Insecure Update Mechanism

An update mechanism must authenticate the update source and verify the integrity and authenticity of new firmware before installation. Otherwise, altered firmware can bypass every protection implemented by the original application.

3.4.1 Exploitable OTA update

Not yet implemented

Preparation

X

Steps to reproduce

1. X

3.4.2 Non-volatile data reading (by altered firmware) - Hard

Once unsigned firmware can be installed, that firmware runs with the same hardware privileges as the original application. It can directly read non-volatile state that was never exposed through the normal interface.

Not yet implemented.

Preparation

X

Steps to reproduce

1. Create a minimal sketch that initializes EEPROM emulation for at least 33 bytes.
2. Read and print address 0, followed by addresses 1 through 32, in hexadecimal and ASCII.
3. Install the altered image through the authorized update or physical flashing procedure.
4. Open the serial monitor and reset the RootNode.
5. Identify the debug flag and reconstruct the stored hostname from the output.
6. Restore the original firmware and filesystem after the exercise.

3.5 Insufficient Privacy Protection

The RootNode does not sufficiently minimize or protect information about its users and network. Credentials can be recovered from the account database, HTTP traffic is unencrypted, and serial commands expose details such as usernames, IP addresses, the MAC address and configuration state. An attacker can combine these data sources to identify users, profile the device and access other functions.

Steps to reproduce

1. Capture a normal login over HTTP and identify the exposed credentials or session information.
2. Use webusers as root and note the disclosed normal-user credentials.
3. Run **ip addr**, **ip route**, **env** in the serial CLI.
4. Download and decrypt *webusers.db* as described in the previous sections.
5. List which pieces of personal, account and network information are available to each role.

3.6 Insecure Data Transfer and Storage

Sensitive RootNode files are stored on SPIFFS and are presented through both the serial file interface and the web file interface. Authorization and path handling must therefore be enforced consistently across all these interfaces.

3.6.1 Exploitable file upload & download - Medium

An authenticated normal user can download externally backed files, including the encrypted web account database. An administrator can upload arbitrary files into /home/user/uploads and can replace reserved backing files. Replacing *webusers.db* immediately reloads the web-user configuration, so stolen administrator access can be made persistent by uploading a correctly formatted encrypted account database.

Preparation

Preserve a copy of the original *webusers.db* before changing it.

Steps to reproduce

1. Sign in as an admin and open the files page.
2. Download */etc/SmartLight/webusers.db*.
3. To add users to the user base, use the leaked AES key and the serial gpg --encrypt command create encrypted username:password:role records.
4. Place the encrypted records below a format: encrypted header and upload the file using the reserved name *webusers.db*.

3.6.2 Path traversal

Not yet implemented

Preparation

Sign in to the file explorer and capture an upload and a download request.

Steps to reproduce

1. XXX an X

3.7 Lack of Device Management

Not yet implemented

3.7.1 X

X

Preparation

X

Steps to reproduce

1. X

3.8 Insecure Default Settings

The firmware contains predictable values such as the smartlight hostname, a recognizable setup SSID and hardcoded serial and root credentials. The serial password is also deliberately leaked during startup. Device-control endpoints are available over HTTP with insufficient authorization. Deploying multiple devices without changing these values gives every unit the same known attack surface.

3.9 Lack of Physical Hardening

Physical access to the RootNode exposes its USB serial interface and ESP8266 bootloader. The enclosure does not provide a trusted boot chain or prevent direct flash access.

3.9.1 Firmware dump - Hard

Not yet fully tested

The ESP8266 ROM bootloader can read external flash when the board is placed in programming mode. Because the RootNode does not use flash encryption or verified boot, a person with physical access can copy the firmware and SPIFFS data for offline analysis. The dump may reveal strings, credentials, cryptographic material and implementation details.

Preparation

Install esptool on an authorized computer and identify the correct COM port. Record the detected flash size before reading it. Keep the RootNode powered from a reliable USB connection during the dump.

Steps to reproduce

1. Connect the RootNode through USB and determine its serial port.
2. Put the board in bootloader mode by holding GPIO0 low while resetting it; use the board-specific flash procedure supplied by the instructor.
3. Run python -m esptool --port <COM-port> flash-id and record the reported flash size. Older esptool versions use the command name flash_id.
4. Read the complete detected range with python -m esptool --port <COM-port> read-flash 0x000000 <flash-size> rootnode-flash.bin. Older versions use read_flash.
5. Calculate a hash of the dump and work on a copy.
6. Inspect the copy with a strings tool, hexadecimal editor or firmware-analysis tool and identify RootNode configuration strings and filesystem content.